

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto: GS Stock & Facturación.

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Link al documento: [Link del documento](#)

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

Resumen

Para esta tarea como equipo nos decantamos por utilizar Katalon Studio como herramienta principal para las pruebas automatizadas de interfaz. Esta plataforma nos permitió grabar y ejecutar los pasos que seguiría un usuario típico al interactuar con la aplicación, facilitando la detección de errores funcionales de manera visual y sin necesidad de tanta complejidad. Su enfoque accesible y su curva de aprendizaje la convirtieron en una solución efectiva para validar la experiencia del usuario y garantizar el correcto funcionamiento de las funcionalidades clave del sistema.

Aunque existen otras herramientas como Selenium, Cypress, Playwright y TestCafe, evaluamos sus características y concluimos que Katalon se adaptaba mejor a las necesidades del proyecto. Por ejemplo, Selenium ofrece gran flexibilidad pero requiere mayor configuración; Cypress es muy rápido pero limitado a ciertos entornos; Playwright es potente pero técnico, y TestCafe es simple, aunque menos visual. En comparación, Katalon proporcionó un equilibrio ideal entre facilidad de uso, potencia y accesibilidad, permitiendo la participación activa de todo el equipo en el proceso de validación, incluso sin experiencia avanzada en testing.

Esta elección ayudó a automatizar escenarios reales de navegación, llenado de formularios y validación visual, mejorando la cobertura de pruebas sin depender de procesos manuales repetitivos. La adopción de Katalon fortaleció la calidad del frontend al detectar posibles fallos antes de su despliegue, asegurando una experiencia de usuario fluida y estable.

Por otro lado, para validar la lógica del backend y las APIs REST, se optó por utilizar Postman como herramienta principal de pruebas. Postman nos permitió construir y ejecutar de forma sencilla una batería de pruebas sobre los endpoints de la API, incluyendo operaciones GET, POST, PUT y DELETE, con distintos escenarios. Además, el uso de variables y colecciones facilitó la automatización de pruebas repetitivas y el manejo de tokens de autenticación.

Plan Maestro:

TC ID#	Escenario / Condición	Rol de Usuario	Acciones	Resultado esperado
CU1	Usuario administrador quiere ver las ventas realizadas o hacer una nueva venta	Administrador	Accede al módulo de ventas, consulta historial o registra nueva venta con datos del cliente y productos	Se muestra el listado de ventas o se registra exitosamente la nueva venta
CU2	Administrador necesita crear un nuevo usuario en el sistema	Administrador	Accede al módulo de usuarios, llena formulario con datos personales y asigna rol	Usuario creado exitosamente con las credenciales y permisos correspondientes
CU3	Administrador requiere manejar información de clientes (añadir o consultar)	Administrador	Ingresa a gestión de clientes, añade nuevo cliente o consulta datos existentes	Información del cliente registrada o consultada correctamente
CU4	Secretaria necesita añadir productos al inventario	Secretaria	Accede al módulo de inventario, llena formulario con datos del producto	Producto agregado exitosamente al inventario con stock inicial
CU5	Usuario vendedor necesita iniciar sesión en el sistema	Vendedor	Llena formulario de login con usuario y contraseña	Acceso exitoso al sistema con permisos de vendedor
CU6	Administrador requiere editar información de productos en inventario	Administrador	Accede a gestión de inventario, selecciona producto y modifica datos	Información del producto actualizada correctamente
CU7	Vendedor necesita procesar un pedido y generar envío	Vendedor	Llena formulario de pedido con datos del cliente y productos, genera orden de envío	Pedido registrado y orden de envío generada exitosamente
CU8	Administrador requiere consultar reportes de ventas por período	Administrador	Accede a módulo de reportes, selecciona rango de fechas y tipo de reporte	Reporte generado con estadísticas de ventas del período seleccionado

CU9	Secretaria necesita actualizar stock de productos existentes	Secretaria	Accede a inventario, busca producto y actualiza cantidades disponibles	Stock del producto actualizado correctamente en el sistema
CU10	Administrador requiere modificar datos de clientes existentes	Administrador	Accede a gestión de clientes, busca cliente y edita información	Datos del cliente actualizados exitosamente
CU11	Vendedor necesita consultar estado de pedidos realizados	Vendedor	Accede a módulo de seguimiento, busca pedido por número o cliente	Se muestra el estado actual del pedido y detalles del envío
CU12	Administrador requiere eliminar usuarios inactivos del sistema	Administrador	Accede a gestión de usuarios, selecciona usuario y confirma eliminación	Usuario eliminado del sistema y permisos revocados

Plan de Pruebas:

Backend

Tabla 1: Modulo de ventas

Caso	Endpoint	Descripción	Datos de Entrada	Respuesta Esperada	HTTP
VP00 1	GET /ventas	Obtener todas las ventas con autenticación válida	Header: Authorization: Bearer <token_válido>	Lista de ventas agrupadas por pedido con estructura completa	200
VP00 2	GET /ventas/:id	Obtener venta específica existente	id: 1, token válido	Datos completos de la venta con productos	200
VP00 3	GET /ventas/:id	Obtener venta inexistente	id: 999, token válido	{ "error": "Venta no encontrada" }	404
VP00 4	GET /clientes/buscar-empresa/:empresa	Buscar clientes por empresa existente	empresa: 'Zapatos', token válido	Lista de clientes de esa empresa	200

VP00 5	POST /ventas	Crear venta válida completa	json { "id_cliente": 1, "id_vendedor": 1, "id_metodo_pago": 1, "productos": [{"id_zapato": 1, "cantidad": 2}], "subtotal": 100.00, "total": 120.00 }	{ "message": "Venta creada exitosamente", "pedido_id": X }	201
VP00 6	POST /ventas	Crear venta con stock insuficiente	Producto con cantidad > stock_disponible	{ "error": "Error al crear la venta", "details": "Stock insuficiente..." }	400
VP00 7	PUT /ventas/:id/estado	Actualizar estado de pedido válido	id: 1, body: { "estado": "Entregado" }, token válido	{ "message": "Estado de pedido actualizado correctamente" }	200
VP00 8	PUT /ventas/:id/estado	Actualizar con estado inválido	id: 1, body: { "estado": "EstadoInvalido" }	{ "error": "Estado de pedido no válido" }	400
VP00 9	POST /inventario/verificar-stock	Verificar stock suficiente	{ "productos": [{"id_zapato": 1, "cantidad": 5}] }	{ "message": "Stock disponible para todos los productos" }	200

[Link al video 1](#)

Tabla 2: Validación creación de usuario

Caso	Endpoint	Descripción	Datos de Entrada	Respuesta Esperada	HTTP
------	----------	-------------	------------------	--------------------	------

VP00 1	POST /usuarios	Crear usuario válido completo	nombre, apellido, email, rol, estado.	Usuario creado correctamente con todos los datos	201
VP00 2	POST /usuarios	Campo obligatorio vacío (email)	nombre, apellido, rol, estado.	El email es obligatorio y no puede estar vacío	400
VP00 3	POST /usuarios	Email duplicado	nombre, apellido, email, rol, estado.	Ya existe un usuario con este email	409
VP00 4	POST /usuarios	Crear usuario inactivo	nombre, apellido, email, rol, estado.	Usuario creado correctamente con estado inactivo	201
VP00 5	POST /usuarios	Campo obligatorio (nombre)	apellido, email, rol, estado.	El nombre es obligatorio y no puede estar vacío	400

[Link Video Test 2](#)

Tabla 3. Prueba de manejo de clientes

TC ID#	Escenario / Condición	Rol de Usuario	Acciones	Resultado esperado
CW1	El Usuario tiene pedidos relacionados	Administrador	Borrar al cliente que tiene pedidos relacionados	Error al eliminar, el cliente tiene pedidos relacionados a su nombre
CW2	Usuario ingresado correctamente	Administrador	LLena los campos de cliente	Los datos del cliente han sido ingresados correctamente

			correctamente	
CW3	Error de permiso	Secretaria	Ingresa datos de un cliente	Error de permiso el usuario no tiene permiso para ingresar clientes
cw4	Cliente eliminado correctamente	Administrador	Eliminar un cliente mal creado en el momento, por lo que no tiene pedidos	Cliente eliminado correctamente

[Link del video 3](#)

Frontend

Tabla 4. Prueba de creación de zapatos

TC ID#	Escenario / Condición	Rol de Usuario	Ingresa	Resultado esperado
CW1	Error de Datos negativos	Administrador	15 zapatos en la tabla de inventario	no se pueden ingresar enteros negativos
CW2	Error de permiso	Vendedor	15 zapatos en la tabla de inventario	Error de permiso, el usuario no tiene permiso para ingresar inventario
CW3	Error de permiso	secretaria	15 zapatos en la tabla de inventario	Error de permiso, el usuario no tiene permiso para ingresar inventario
CW3	Datos ingresados correctamente	Administrador	15 zapatos en la tabla de inventario	Los datos se han ingresado correctamente
CW4	Datos = 0	Administrador	0 zapatos en la tabla de inventario	No se puede ingresar zapatos < 1

[Link del video 4](#)

Tabla 5. Validación de formulario de inicio de sesión

TC ID#	Escenario / Condición	Rol de Usuario	Acciones	Resultado esperado
CW1	Campo obligatorio vacío	Administrador	Campo “Usuario” vacío	Muestra error: “Completa este campo”
CW2	Credenciales incorrectas	Administrador	Contraseña o Usuario incorrecto	Credenciales inválidas
CW3	Datos de acceso válidos	Administrador	Campos correctos	Inicio de sesión exitoso
cw4	Campo de contraseña vacía	Administrador	Llena todos los campos menos el campo de contraseña	Completa este campo

[Link video 5](#)

Tabla 6. Validación de la vista “Gestión de Inventarios”

TC ID#	Escenario / Condición	Rol de Usuario	Acciones	Resultado esperado
CW1	Botón “Seleccionar” funcional	Administrador	Presionar el botón “Seleccionar” de cualquier fila	Se activa la opción “Editar inventario” y “Eliminar inventario”
CW2	Botón Editar inventario	Administrador	Presionar editar inventario	Permite modificar Cantidad y Estado.
CW3	Búsqueda por ID inexistente	Administrador	Ingresar un ID inexistente en el campo de búsqueda	La tabla de inventario queda vacía.
cw4	Carga completa de la tabla	Administrador	Entrar a la vista “Inventario” desde el menú principal	La tabla muestra los registros existentes con columnas: ID, Cantidad, Estado, etc.

[Link Video 6](#)

Tabla 7. Prueba de manejo de clientes

TC ID#	Escenario / Condición	Rol de Usuario	Acciones	Resultado esperado
CW1	El Usuario tiene pedidos relacionados	Administrador	Borrar al cliente que tiene pedidos relacionados	Error al eliminar, el cliente tiene pedidos relacionados a su nombre
CW2	Usuario ingresado correctamente	Administrador	Llena los campos de cliente correctamente	Los datos del cliente han sido ingresados correctamente
CW3	Error de permiso	Secretaria	Ingresa datos de un cliente	Error de permiso el usuario no tiene permiso para ingresar clientes
cw4	Cliente eliminado correctamente	Administrador	Eliminar un cliente mal creado en el momento, por lo que no tiene pedidos	Cliente eliminado correctamente

[Link del video 7](#)

Conclusiones

Usar herramientas de pruebas automatizadas fue muy útil para mejorar nuestro sistema. Katalon Recorder nos ayudó a probar la parte visual del sistema frontend de forma sencilla, grabando los pasos que hace un usuario. Esto nos permitió encontrar errores fácilmente sin tener que escribir mucho código, y todo el equipo pudo participar, aunque no tuviera experiencia en pruebas. También usamos Postman para probar la parte del servidor backend y las APIs. Con Postman pudimos enviar peticiones como GET, POST, PUT y DELETE para ver si el sistema respondía bien en diferentes situaciones. Además, su uso de variables y colecciones nos permitió repetir pruebas fácilmente y asegurarnos de que el sistema respondiera correctamente, incluso cuando algo salía mal.

Referencias

Katalon. (s.f.). Katalon Recorder overview. Katalon Platform. Recuperado el 4 de agosto de 2025, de <https://docs.katalon.com/katalon-platform/plugins-and-add-ons/katalon-recorder-extension/get-started/katalon-recorder-overview>

OpenWebinars. (2021, mayo 6). ¿Qué es Postman y para qué sirve? OpenWebinars. <https://openwebinars.net/blog/que-es-postman/>